LECTURE 5

Pandas, Part III and EDA, Part I

Advanced Pandas (More on Grouping, Aggregation, Pivot Tables, and Merging)

Exploratory Data Analysis and its role in the data science lifecycle

Presented by Dr. Devan Becker, Data200

Adapted from Data 100, Summer 2023 @ UC Berkeley by Bella Crouch and Dominic Liu

Content credit: Lisa Yan, Josh Hug, Suraj Rampure, Allen Shen, Joey Gonzalez, and Sam Lau

CC BY NC SA

Announcements

- Course notes link has changed
  - The Berkely team is updating their notes.
  - Add "_su23" to the last part of the URL
- There is a lab this week
- A2 due TONIGHT!
- Office hours tomorrow from 10-12

CC BY NC SA

Today’s Roadmap

- Pandas, Part III
  - More on Groupby
  - Pivot Tables
  - Joining Tables
- EDA, Part I
  - Structure: Tabular Data
  - Granularity
  - Structure: Variable Types
  - Structure: Multiple Files

CC BY NC SA

More on Groupby

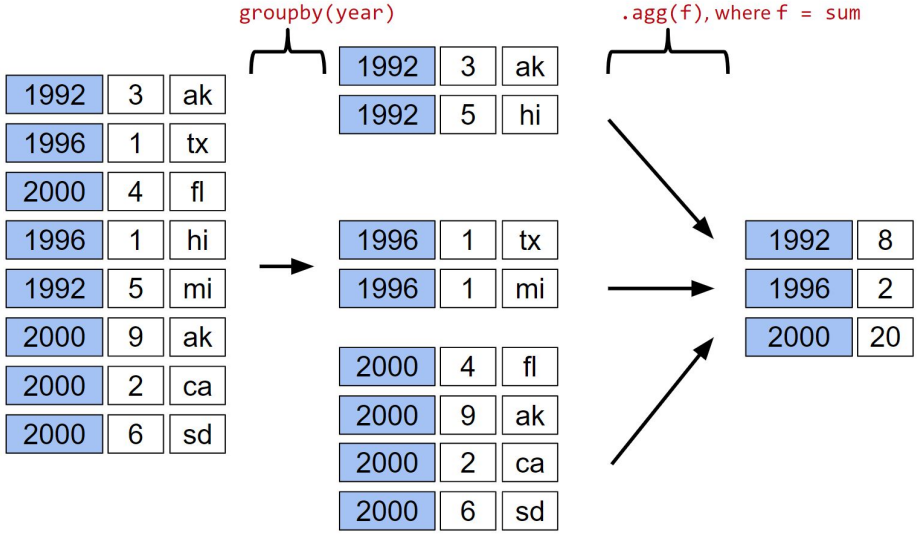
- **Pandas, Part III**
  - **More on Groupby**
  - Pivot Tables
  - Joining Tables
- EDA, Part I
  - Structure: Tabular Data
  - Granularity
  - Structure: Variable Types
  - Multiple Files

CC BY NC SA

Revisiting `groupby.agg`

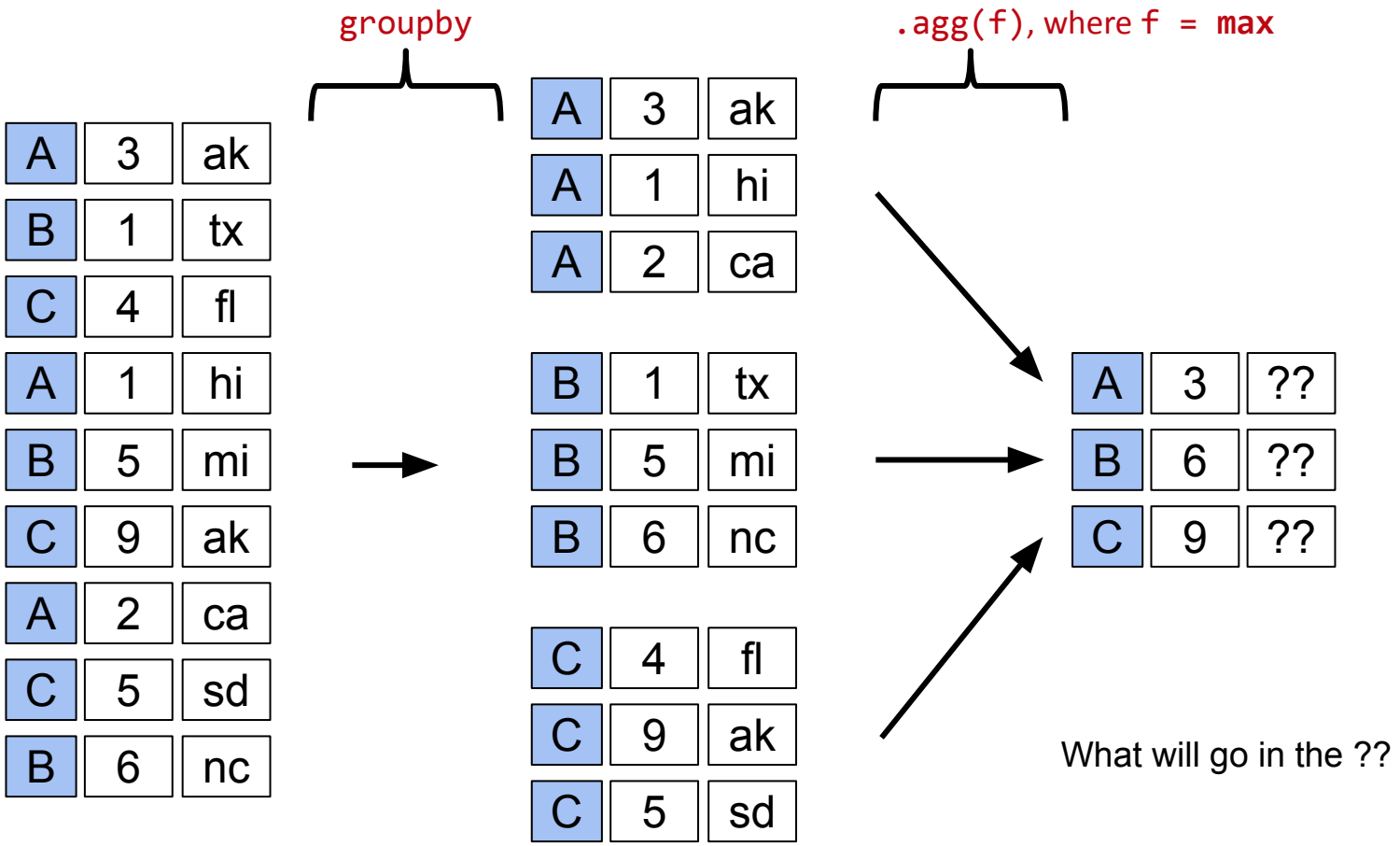
A `groupby` operation involves some combination of **splitting the object, applying a function**, and **combining the results**.

- So far, we've seen that `df.groupby("year").agg(sum)`:
  - **Split** `df` into sub-DataFrames based on `year`
  - **Apply** the `sum` function to each column of each sub-DataFrame
  - **Combine** the results of `sum` into a single DataFrame, indexed by `year`



CC BY NC SA

Groupby Review Question



CC BY NC SA

Raw `GroupBy` Objects and other methods

The result of a groupby operation applied to a DataFrame is a **`DataFrameGroupBy`** object.

- It is not a DataFrame!

```
grouped_by_year = elections.groupby("Year")
type(grouped_by_year)
```

```
pandas.core.groupby.generic.DataFrameGroupBy
```

Given a **`DataFrameGroupBy`** object, can use various functions to generate **`DataFrames`** (or **`Series`**). **`agg`** is only one choice:

```
df.groupby(col).mean()
df.groupby(col).sum()
df.groupby(col).min()
df.groupby(col).max()
```

```
df.groupby(col).first()
df.groupby(col).last()
df.groupby(col).size()
df.groupby(col).count()
```

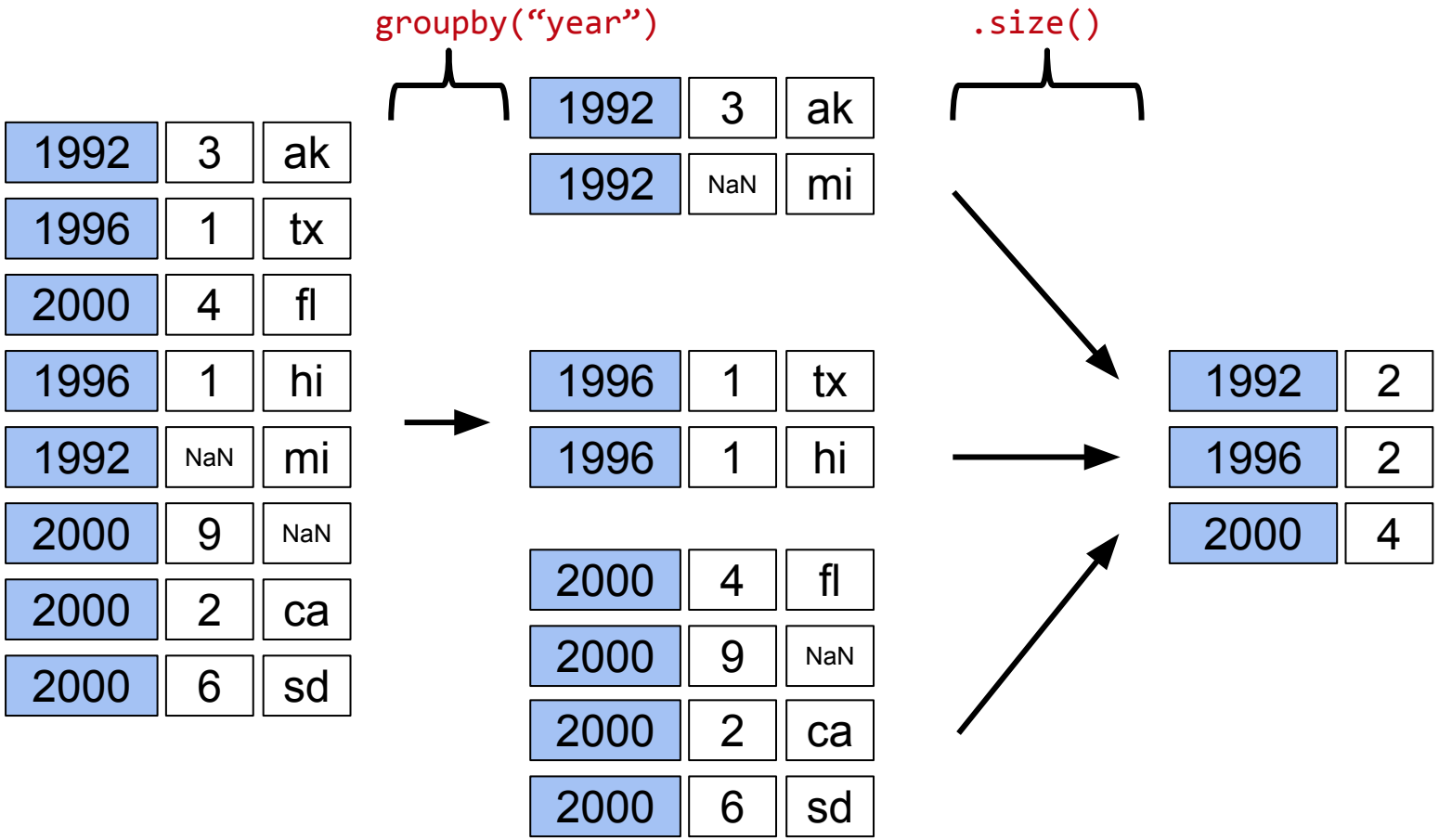
```
df.groupby(col).filter()
```

🤔 What's the difference?

See https://pandas.pydata.org/docs/reference/groupby.html for a list of DataFrameGroupBy methods.

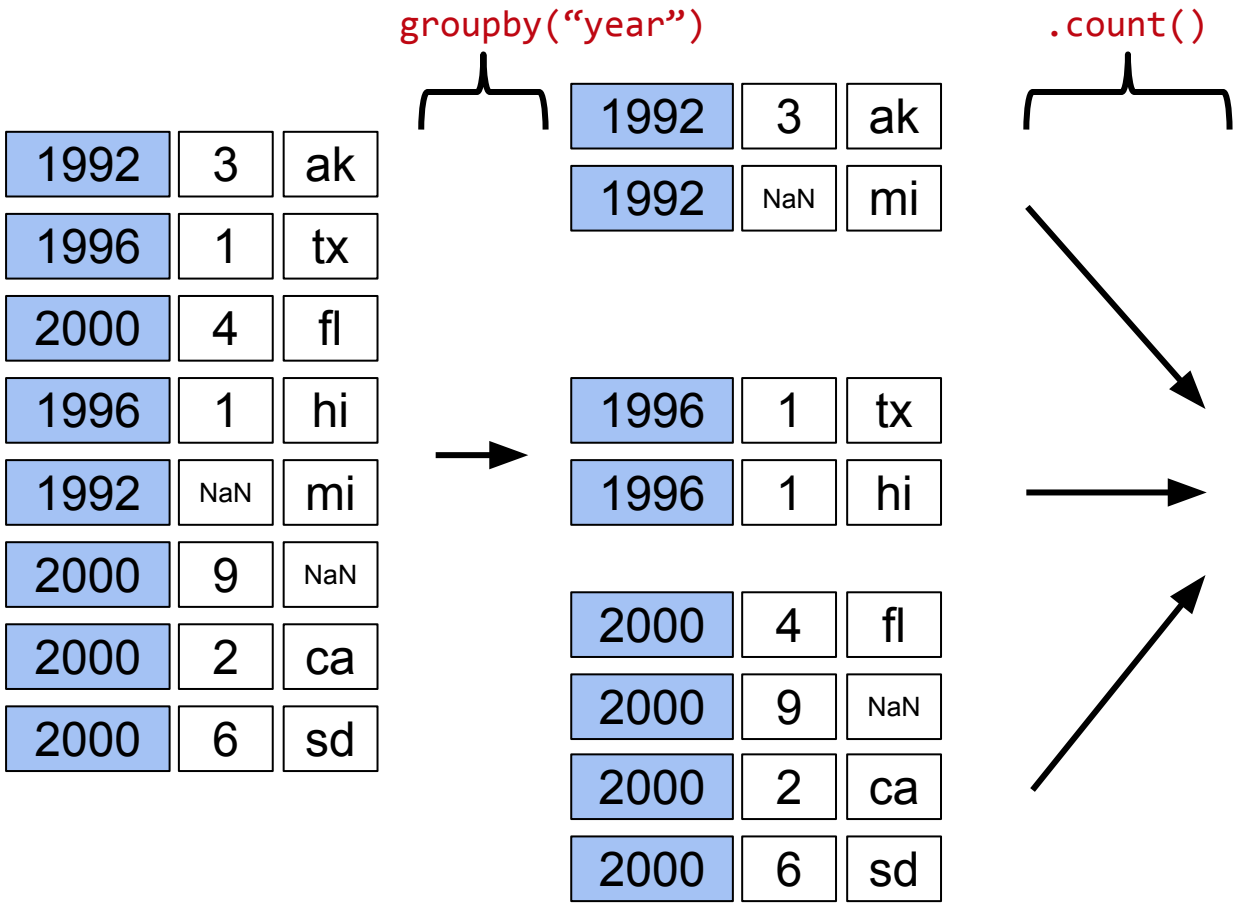
CC BY NC SA

groupby.size() and groupby.count()



CC BY NC SA

groupby.size() and groupby.count()



CC BY NC SA

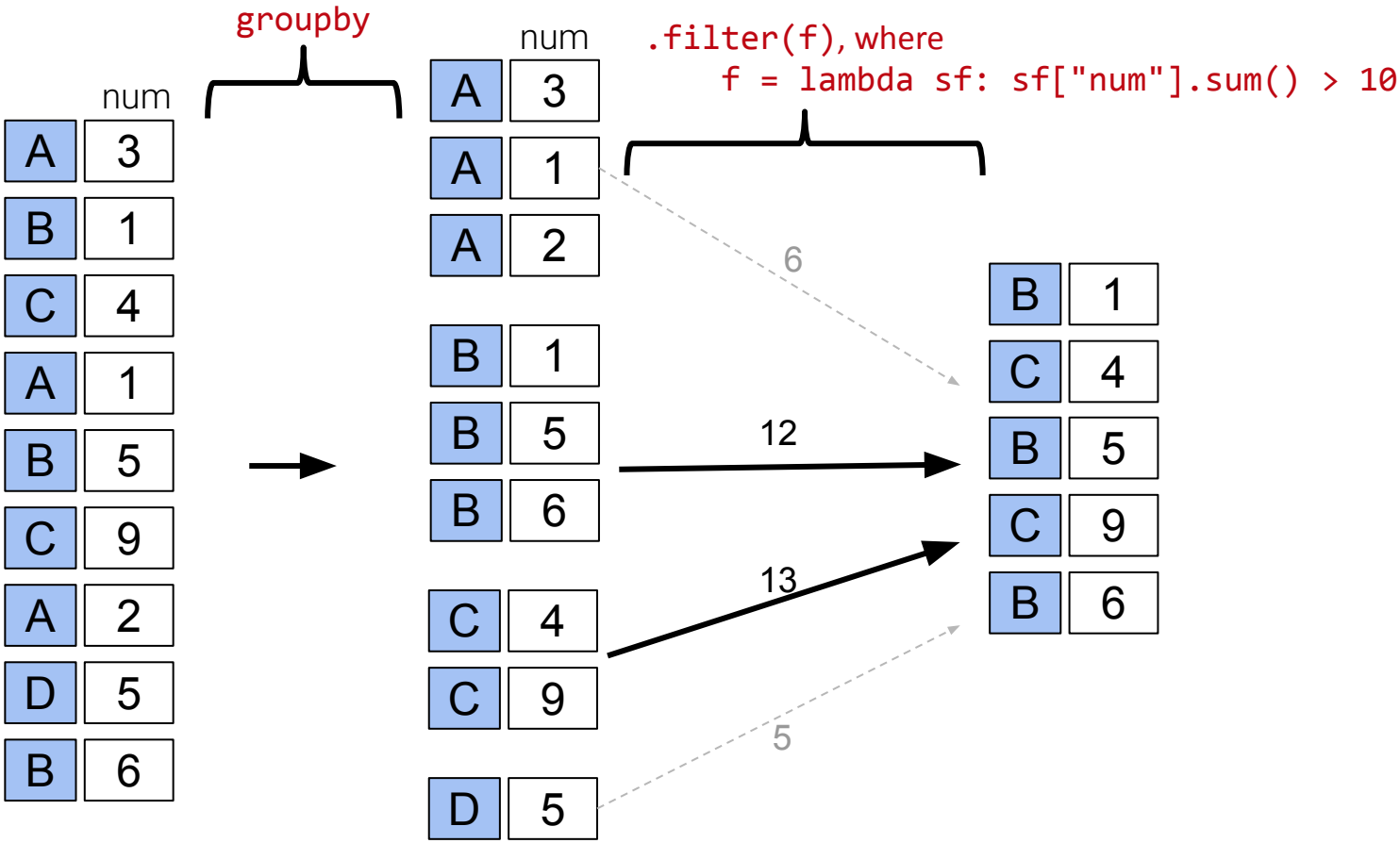
Filtering by Group

Another common use for groups is to filter data.

- `groupby.filter` takes an argument `func`.
- `func` is a function that:
  - Takes a DataFrame as input.
  - Returns either True or False.
- `filter` applies `func` to each group/sub-DataFrame:
  - If `func` returns **True** for a group, then all rows belonging to the group are **preserved**
  - If `func` returns **False** for a group, then all rows belonging to that group are **filtered out**
- Notes
  - Filtering is done per group, not per row. Different from boolean filtering.
  - Unlike `agg()`, the column we grouped on does NOT become the index!

CC BY NC SA

groupby.filter()



CC BY NC SA

Filtering Elections Dataset

Going back to the `elections` dataset.

Let's keep only election year results where the max '%' is less than 45%.

```
elections.groupby("Year").filter(lambda sf: sf["%"].max() < 45)
```

	Year	Candidate	Party	Popular vote	Result	%
23	1860	Abraham Lincoln	Republican	1855993	win	39.699408
24	1860	John Bell	Constitutional Union	590901	loss	12.639283
25	1860	John C. Breckinridge	Southern Democratic	848019	loss	18.138998
26	1860	Stephen A. Douglas	Northern Democratic	1380202	loss	29.522311
66	1912	Eugene V. Debs	Socialist	901551	loss	6.004354
67	1912	Eugene W. Chafin	Prohibition	208156	loss	1.386325
68	1912	Theodore Roosevelt	Progressive	4122721	loss	27.457433
69	1912	William Taft	Republican	3486242	loss	23.218466
70	1912	Woodrow Wilson	Democratic	6296284	win	41.933422
115	1968	George Wallace	American Independent	9901118	loss	13.571218

CC BY NC SA

groupby puzzle

Puzzle: We want to know the **best election by each party**

- Best election: the election with the highest % of votes
- For example, Democrat's best election was in 1964, with candidate Lyndon Johnson winning 61.3% of votes.

Party	Year	Candidate	Popular vote	Result	%
American	1856	Millard Fillmore	873053	loss	21.554001
American Independent	1968	George Wallace	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2008	Chuck Baldwin	199750	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	1964	Lyndon Johnson	43127041	win	61.344703

CC BY NC SA

Attempt #1

Why does the table seem to claim that Woodrow Wilson won the presidency in 2020?

```
elections.groupby("Party").max().head(10)
```

Party	Year	Candidate	Popular vote	Result	%
American	1976	Thomas J. Anderson	873053	loss	21.554001
American Independent	1976	Lester Maddox	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2016	Michael Peroutka	203091	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	2020	Woodrow Wilson	81268924	win	61.344703
Democratic-Republican	1824	John Quincy Adams	151271	win	57.210122

CC BY NC SA

Problem with Attempt #1

Why does the table seem to claim that Woodrow Wilson won the presidency in 2020?

```
elections.groupby("Party").max().head(10)
```

Party	Year	Candidate	Popular vote	Result	%
American	1976	Thomas J. Anderson	873053	loss	21.554001
American Independent	1976	Lester Maddox	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2016	Michael Peroutka	203091	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	2020	Woodrow Wilson	81268924	win	61.344703
Democratic-Republican	1824	John Quincy Adams	151271	win	57.210122

Every column is calculated independently! Among Democrats:

- Last year they ran: 2020
- Alphabetically the latest candidate name: Woodrow Wilson
- Highest % of vote: 61.34

CC BY NC SA

Attempt #2: Motivation

- We want to preserve entire rows, so we need an aggregate function that does that

Party	Year	Candidate	Popular vote	Result	%
American	1856	Millard Fillmore	873053	loss	21.554001
American Independent	1968	George Wallace	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2008	Chuck Baldwin	199750	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	1964	Lyndon Johnson	43127041	win	61.344703

CC BY NC SA

Attempt #2: Solution

- First sort the DataFrame so that rows are in descending order of %.
- Then group by Party and take the first item of each sub-DataFrame.
- Note: Lab will give you a chance to try this out if you didn't quite follow during lecture.

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)
elections_sorted_by_percent.groupby("Party").first()
```

	Year	Candidate	Party	Popular vote	Result	%
114	1964	Lyndon Johnson	Democratic	43127041	win	61.344703
91	1936	Franklin Roosevelt	Democratic	27752648	win	60.978107
120	1972	Richard Nixon	Republican	47168710	win	60.907806
79	1920	Warren Harding	Republican	16144093	win	60.574501
133	1984	Ronald Reagan	Republican	54455472	win	59.023326

```
elections_sorted_by_percent
```

Party	Year	Candidate	Popular vote	Result	%
American	1856	Millard Fillmore	873053	loss	21.554001
American Independent	1968	George Wallace	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2008	Chuck Baldwin	199750	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	1964	Lyndon Johnson	43127041	win	61.344703

CC BY NC SA

Attempt #2: Solution

`.sort_values("%", ascending = False)`

`.groupby("Party")`

`.first()`

Order is preserved in sub-dataframes!

DR	1824	57%
DR	1824	43%
Dem	1828	56%
Nat	1828	44%
Dem	1832	54%
…		
Dem	2020	51%
Rep	2020	47%
Green	2020	0.2%

Dem	1964	61%
Dem	1936	60%
Rep	1972	60%
Rep	1920	60%
Rep	1984	59%
…		
Cons	2004	0.1%
Pop	1992	0.1%
Green	2004	0.01%

Dem	1964	61%
Dem	1936	60%
Rep	1972	60%
Rep	1920	60%
Rep	1984	59%
Green	2020	0.2%
Green	2004	0.01%

Dem	1964	61%
Rep	1972	60%
Green	2000	2.7%

CC BY NC SA

groupby puzzle - Alternate Approaches

Using a **lambda** function

```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)
elections_sorted_by_percent.groupby("Party").agg(lambda x : x.iloc[0])
```

Using **idxmax** function

```
best_per_party = elections.loc[elections.groupby("Party")["%"].idxmax()]
```

Using **drop_duplicates** function

```
best_per_party2 = elections.sort_values("%").drop_duplicates(["Party"], keep="last")
```


CC BY NC SA

There's More Than One Way to Find the Best Result by Party

In Pandas, there's more than one way to get to the same answer.

- Each approach has different tradeoffs in terms of readability, performance, memory consumption, complexity, etc.
- Takes a very long time to understand these tradeoffs!
- If you find your current solution to be particularly convoluted or hard to read, maybe try finding another way!

CC BY NC SA

More on `DataFrameGroupby` Object

We can look into **`DataFrameGroupby`** objects in following ways:

```
grouped_by_party = elections.groupby("Party")
grouped_by_party.groups
```

```
{'American': [22, 126], 'American Independent': [115, 119, 124], 'Anti-Masonic': [6], 'Anti-Monopoly': [38], 'Citiz
ens': [127], 'Communist': [89], 'Constitution': [160, 164, 172], 'Constitutional Union': [24], 'Democratic': [2, 4,
8, 10, 13, 14, 17, 20, 28, 29, 34, 37, 39, 45, 47, 52, 55, 57, 64, 70, 74, 77, 81, 83, 86, 91, 94, 97, 100, 105, 10
8, 111, 114, 116, 118, 123, 129, 134, 137, 140, 144, 151, 158, 162, 168, 176, 178], 'Democratic-Republican': [0,
1], 'Dixiecrat': [103], 'Farmer-Labor': [78], 'Free Soil': [15, 18], 'Green': [149, 155, 156, 165, 170, 177, 181],
'Greenback': [35], 'Independent': [121, 130, 143, 161, 167, 174], 'Liberal Republican': [31], 'Libertarian': [125,
128, 132, 138, 139, 146, 153, 159, 163, 169, 175, 180], 'National Democratic': [50], 'National Republican': [3, 5],
'National Union': [27], 'Natural Law': [148], 'New Alliance': [136], 'Northern Democratic': [26], 'Populist': [48,
61, 141], 'Progressive': [68, 82, 101, 107], 'Prohibition': [41, 44, 49, 51, 54, 59, 63, 67, 73, 75, 99], 'Reform':
[150, 154], 'Republican': [21, 23, 30, 32, 33, 36, 40, 43, 46, 53, 56, 60, 65, 69, 72, 79, 80, 84, 87, 90, 96, 98,
104, 106, 109, 112, 113, 117, 120, 122, 131, 133, 135, 142, 145, 152, 157, 166, 171, 173, 179], 'Socialist': [58, 6
2, 66, 71, 76, 85, 88, 92, 95, 102], 'Southern Democratic': [25], 'States' Rights': [110], 'Taxpayers': [147], 'Uni
on': [93], 'Union Labor': [42], 'Whig': [7, 9, 11, 12, 16, 19]}
```

```
grouped_by_party.get_group("Socialist")
```

	Year	Candidate	Party	Popular vote	Result	%
58	1904	Eugene V. Debs	Socialist	402810	loss	2.985897
62	1908	Eugene V. Debs	Socialist	420852	loss	2.850866
66	1912	Eugene V. Debs	Socialist	901551	loss	6.004354
71	1916	Allan L. Benson	Socialist	590524	loss	3.194193

CC BY NC SA

Pivot Tables

- **Pandas, Part III**
  - More on Groupby
  - **Pivot Tables**
  - Joining Tables
- EDA, Part I
  - Structure: Tabular Data
  - Granularity
  - Structure: Variable Types
  - Multiple Files

CC BY NC SA

Grouping by Multiple Columns

Suppose we want to build a table showing the total number of babies born of each sex in each year. One way is to **`groupby`** *using both columns* of interest:

```
babynames.groupby(["Year", "Sex"]).agg(sum).head(6)
```

Year	Sex	Count
1880	F	90994
	M	110490
1881	F	91953
	M	100737
1882	F	107847
	M	113686

Note: Resulting DataFrame is multi-indexed. That is, its index has multiple dimensions. Will explore in a later lecture.

CC BY NC SA

Pivot Tables

A more natural approach is to create a pivot table.

```
babynames_pivot = babynames.pivot_table(
    index = "Year",      # rows (turned into index)
    columns = "Sex",     # column values
    values = ["Count"], # field(s) to process in each group
    aggfunc = np.sum     # group operation
)
babynames_pivot.head(6)
```

	Count	
Sex	**F**	**M**
Year		
1880	90994	110490
1881	91953	100737
1882	107847	113686
1883	112319	104625
1884	129019	114442
1885	133055	107799

CC BY NC SA

groupby(["Year", "Sex"]) vs. pivot_table

The pivot table more naturally represents our data.

`groupby` output

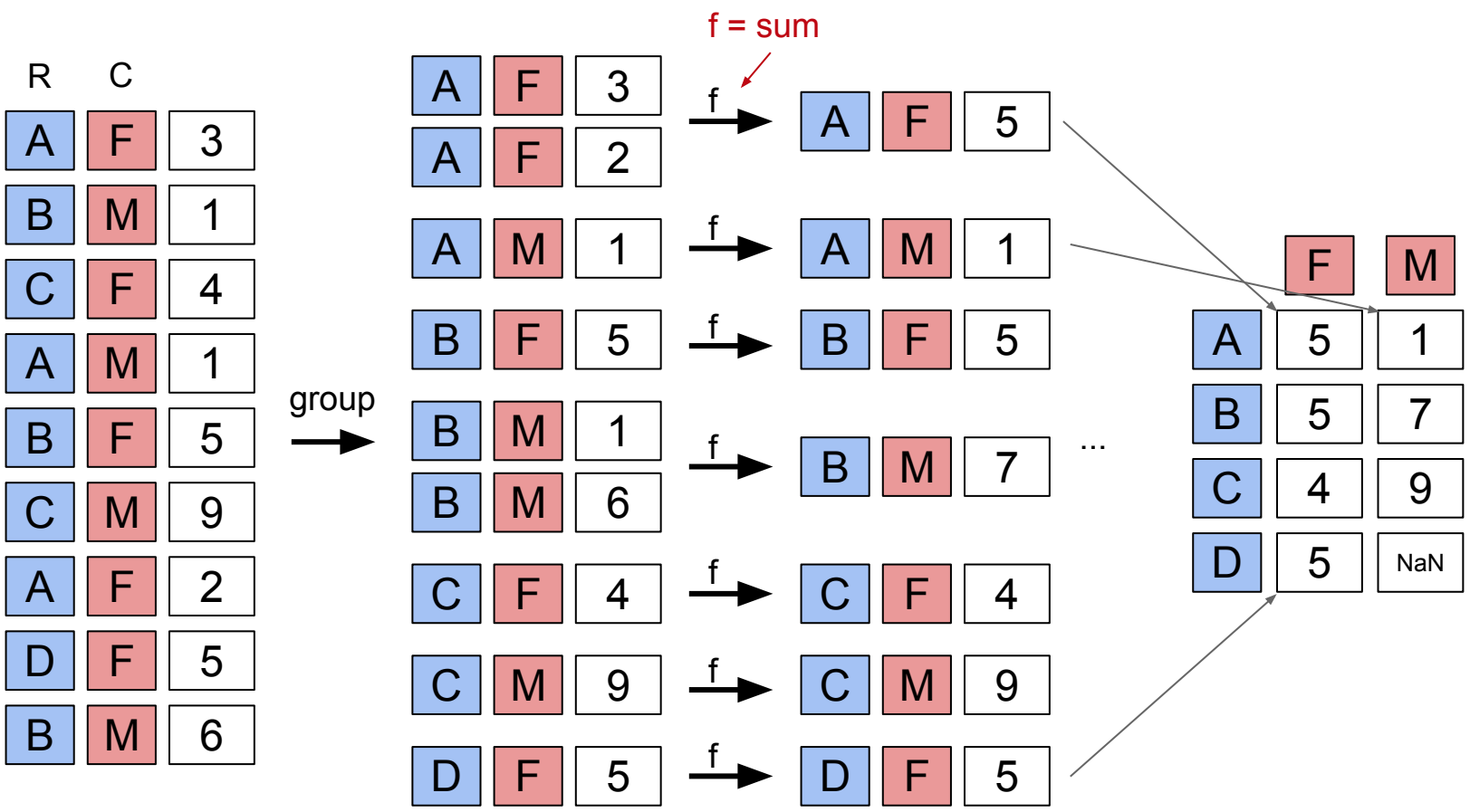
		Count
Year	**Sex**	
1880	**F**	90994
	M	110490
1881	**F**	91953
	M	100737
1882	**F**	107847
	M	113686

`pivot_table` output

	Count	
Sex	**F**	**M**
Year		
1880	90994	110490
1881	91953	100737
1882	107847	113686
1883	112319	104625
1884	129019	114442
1885	133055	107799

CC BY NC SA

Pivot Table Mechanics



CC BY NC SA

Pivot Tables with multiple values

We can include multiple values in our pivot tables.

```
babynames_pivot = babynames.pivot_table(
    index="Year",      # rows (turned into index)
    columns="Sex",     # column values
    values=["Count", "Name"],
    aggfunc=np.max,    # group operation
)
babynames_pivot.head(6)
```

	Count		Name	
Sex	**F**	**M**	**F**	**M**
Year				
1910	295	237	Yvonne	William
1911	390	214	Zelma	Willis
1912	534	501	Yvonne	Woodrow
1913	584	614	Zelma	Yoshio
1914	773	769	Zelma	Yoshio
1915	998	1033	Zita	Yukio

CC BY NC SA

Join Tables

- **Pandas, Part III**
  - More on Groupby
  - Pivot Tables
  - **Joining Tables**
- EDA, Part I
  - Structure: Tabular Data
  - Granularity
  - Structure: Variable Types
  - Multiple Files

CC BY NC SA

Joining Tables

Suppose want to know the popularity of presidential candidate's names in 2020.

- Example: Dwight Eisenhower's name Dwight is not popular today, with only 5 babies born with this name in California in 2020.

BY NC SA

Creating Table 1: babynames in 2020

Let's set aside names from 2020 first:

```
babynames_2020 = babynames[babynames["Year"] == 2020]
babynames_2020
```

	Name	Sex	Count	Year
0	Olivia	F	17641	2020
1	Emma	F	15656	2020
2	Ava	F	13160	2020
3	Charlotte	F	13065	2020
4	Sophia	F	13036	2020
...	...	...	...	...
31448	Zykell	M	5	2020
31449	Zylus	M	5	2020
31450	Zymari	M	5	2020
31451	Zyn	M	5	2020
31452	Zyran	M	5	2020

31453 rows × 4 columns

CC BY NC SA

Creating Table 2: Presidents with First Names

To join our table, we'll also need to set aside the first names of each candidate.

- By default, `str.split()` splits a string on white space.

```
elections["First Name"] = elections["Candidate"].str.split().str[0]
```

	Year	Candidate	Party	Popular vote	Result	%	First Name
...	...	...	...	...	...	...	...
177	2016	Jill Stein	Green	1457226	loss	1.073699	Jill
178	2020	Joseph Biden	Democratic	81268924	win	51.311515	Joseph
179	2020	Donald Trump	Republican	74216154	loss	46.858542	Donald
180	2020	Jo Jorgensen	Libertarian	1865724	loss	1.177979	Jo
181	2020	Howard Hawkins	Green	405035	loss	0.255731	Howard

CC BY NC SA

Joining Our Tables: two options

```
merged = pd.merge(left = elections, right = babynames_2020,
                  left_on = "First Name", right_on = "Name")

merged = elections.merge(right = babynames_2020,
                  left_on = "First Name", right_on = "Name")
```

	Year_x	Candidate	Party	Popular vote	Result	%	First Name	State	Sex	Year_y	Name	Count
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	Andrew	CA	M	2020	Andrew	867
1	1828	Andrew Jackson	Democratic	642806	win	56.203927	Andrew	CA	M	2020	Andrew	867
2	1832	Andrew Jackson	Democratic	702735	win	54.574789	Andrew	CA	M	2020	Andrew	867
3	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	John	CA	M	2020	John	617
4	1828	John Quincy Adams	National Republican	500897	loss	43.796073	John	CA	M	2020	John	617

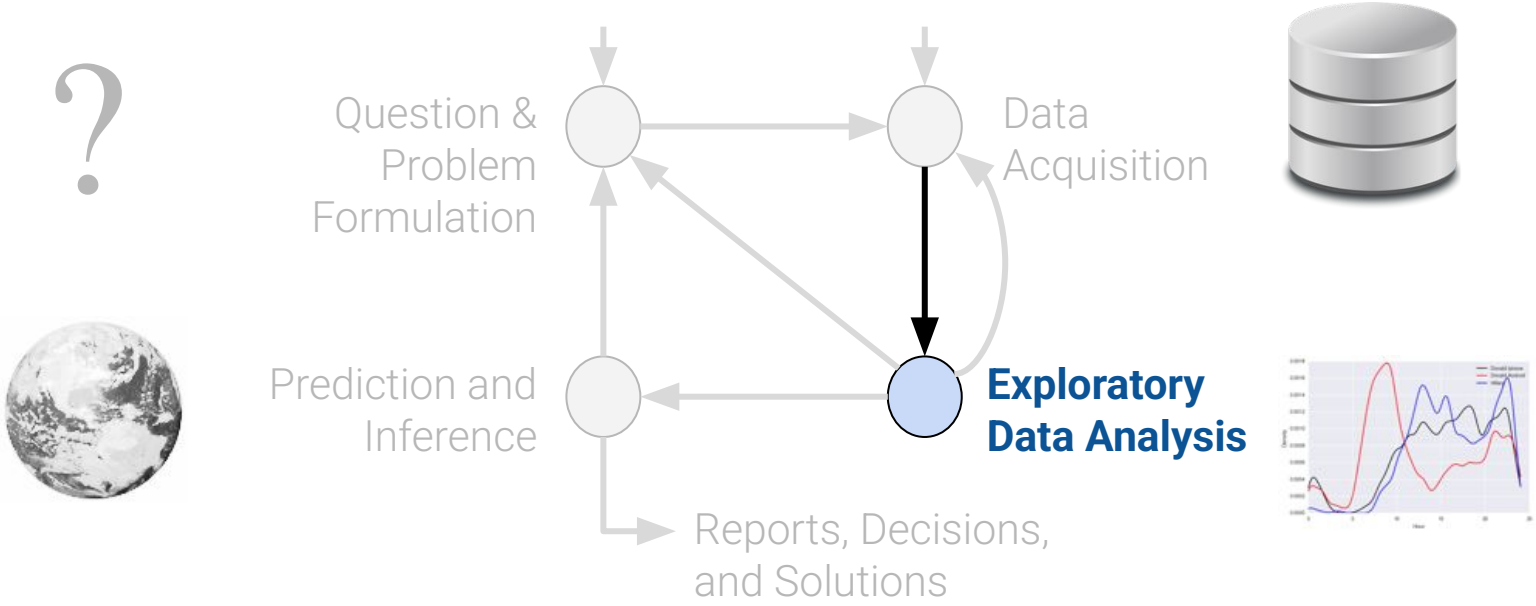
CC BY NC SA

The Next Step

EDA Guiding Principles

CC BY NC SA

Plan for the coming week



(Week 1)

Exploring and Cleaning Tabular Data

From `datascience` to `pandas`

(Week 2)

Data Science in Practice

EDA, Data Cleaning, Text processing (regular expressions)

CC BY NC SA

Key Data Properties to Consider in EDA

Structure -- the “shape” of a data file

Granularity -- how fine/coarse is each datum

Scope -- how (in)complete is the data

Temporality -- how is the data situated in time

Faithfulness -- how well does the data capture “reality”

CC BY NC SA

Tabular Data

- Pandas, Part III
  - More on Groupby
  - Pivot Tables
  - Joining Tables
- **EDA, Part I**
  - **Structure: Tabular Data**
  - Granularity
  - Structure: Variable Types
  - Multiple Files

CC BY NC SA

Key Data Properties to Consider in EDA

File Format
Variable Type
Multiple files
(Primary and Foreign Keys)

Structure -- the “shape” of a data file

Granularity -- how fine/coarse is each datum

Scope -- how (in)complete is the data

Temporality -- how is the data situated in time

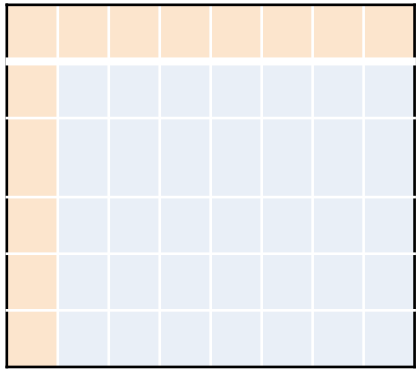
Faithfulness -- how well does the data capture “reality”

CC BY NC SA

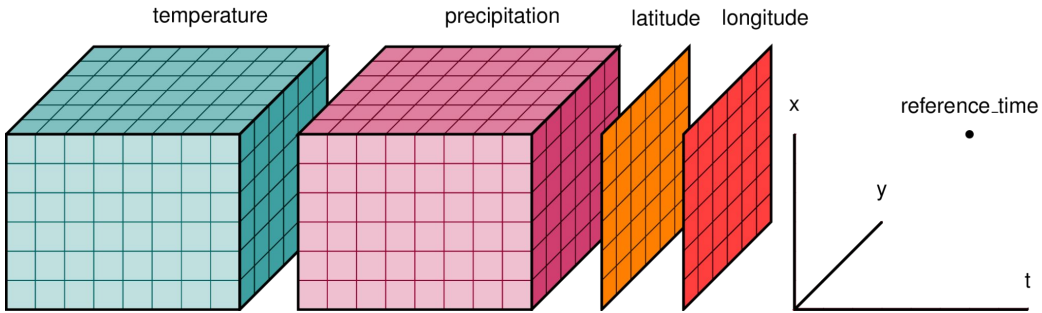
Rectangular and Non-rectangular Data

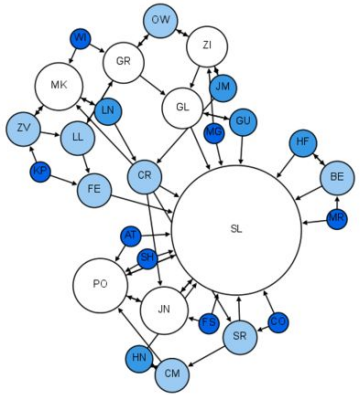
Data come in many different shapes.

Rectangular data



Non-rectangular data



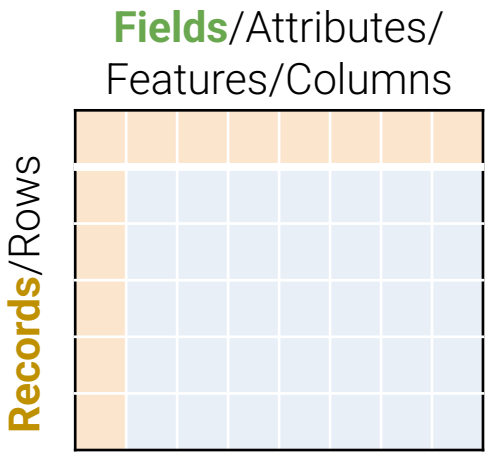


CC BY NC SA

Rectangular Data

We prefer rectangular data for data analysis:

- Regular structures are easy to manipulate and analyze
- A big part of data cleaning is about transforming data to be more rectangular



Two kinds of rectangular data: **Tables** and **Matrices**.

Tables (a.k.a. dataframes in R/Python and relations in SQL)

- Named columns with different types
- Manipulated using data transformation languages (map, filter, group by, join, …)

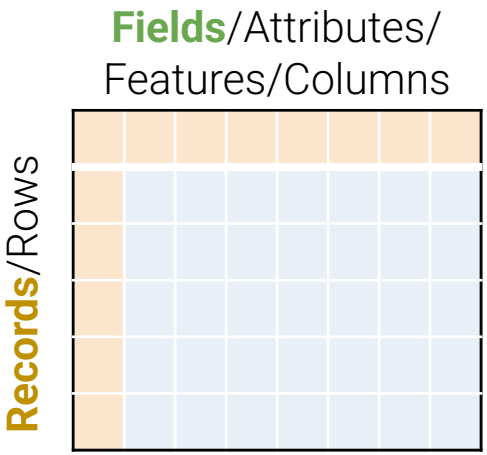
Matrices

- Numeric data of the same type (float, int, etc.)
- Manipulated using linear algebra

CC BY NC SA

Tidy Data

- Each row represents an individual.
  - Where measurements are made.
- Each column represents a feature.



Tidy

	Count	
Sex	F	M
Year		
1880	90994	110490
1881	91953	100737
1882	107847	113686
1883	112319	104625
1884	129019	114442
1885	133055	107799

Not Tidy

		Count
Year	Sex	
1880	F	90994
	M	110490
1881	F	91953
	M	100737
1882	F	107847
	M	113686

CC BY NC SA

Tuberculosis – United States, 2021

Summary

What is already known about this topic?

The number of reported U.S. tuberculosis (TB) cases decreased sharply in 2020, possibly related to multiple factors associated with the COVID-19 pandemic.

What is added by this report?

Reported TB incidence (cases per 100,000 persons) increased 9.4%, from 2.2 during 2020 to 2.4 during 2021 but was lower than incidence during 2019 (2.7). Increases occurred among both U.S.-born and non–U.S.-born persons.

What are the implications for public health practice?

Factors contributing to changes in reported TB during 2020–2021 likely include an actual reduction in TB incidence as well as delayed or missed TB diagnoses. Timely evaluation and treatment of TB and latent tuberculosis infection remain critical to achieving U.S. TB elimination.

CDC Morbidity and Mortality Weekly Report (MMWR) 03/25/2022.

What is **incidence**? Why use it here?

How was "9.4% increase" computed?

Question: Can we **reproduce** these numbers using government data?

CDC: source, 2021

CC BY NC SA

Demo Slides

CSV: Comma-Separated Values

Tuberculosis in the US [CDC source].

CSV is a very common **tabular file format**.

- **Records** (rows) are delimited by a newline: `'\n'`, `"\r\n"`
- **Fields** (columns) are delimited by commas: `','`

Pandas: `pd.read_csv(header=...)`

Fields/Attributes/Features/Columns

Records/Rows

	U.S. jurisdiction	TB cases 2019	…
0	Total	8,900	…
1	Alabama	87	…

CC BY NC SA

Granularity

- Pandas, Part III
  - More on Groupby
  - Pivot Tables
  - Joining Tables
- **EDA, Part I**
  - Structure: Tabular Data
  - **Granularity**
  - Structure: Variable Types
  - Multiple Files

CC BY NC SA

(we'll come back to this)

Key Data Properties to Consider in EDA

Structure -- the "shape" of a data file

Granularity -- how fine/coarse is each datum

Scope -- how (in)complete is the data

Temporality -- how is the data situated in time

Faithfulness -- how well does the data capture "reality"

CC BY NC SA

Granularity: How fine/coarse is each datum?

What does each **record** represent?

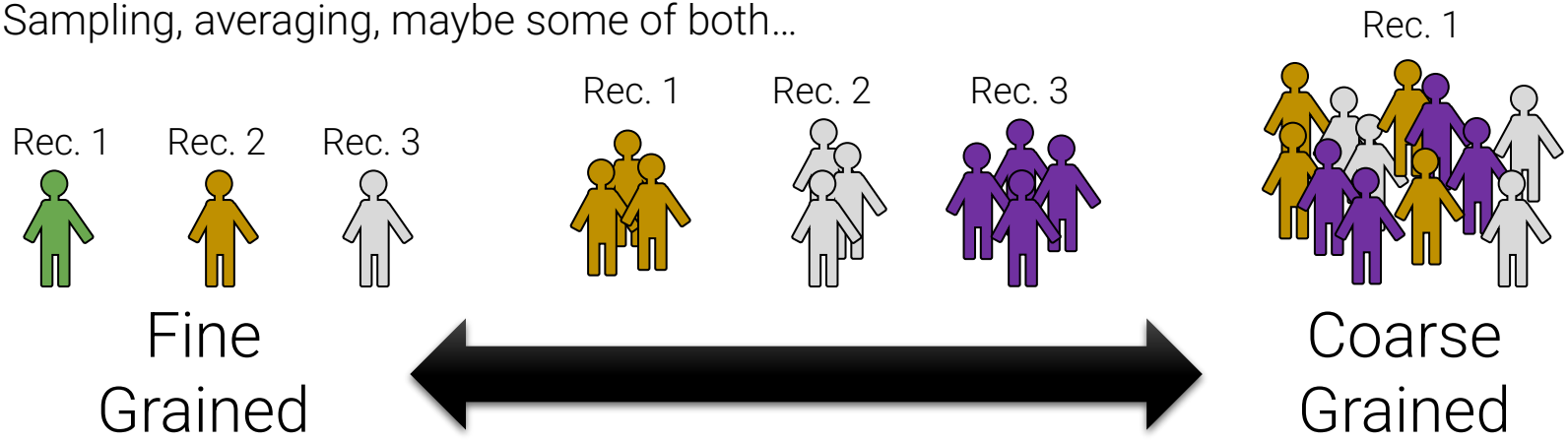
- Examples: a purchase, a person, a group of users

Do all records capture granularity at the same level?

- Some data will include summaries (aka **rollups**) as records.

If the data are **coarse**, how were the records aggregated?

- Sampling, averaging, maybe some of both…



CC BY NC SA

Variable Types

- Pandas, Part III
  - More on Groupby
  - Pivot Tables
  - Joining Tables
- **EDA, Part I**
  - Structure: Tabular Data
  - Granularity
  - **Structure: Variable Types**
  - Multiple Files

CC BY NC SA

Key Data Properties to Consider in EDA

(we're back to this)

File Format

Variable Type

Multiple files
(Primary and Foreign Keys)

Structure -- the "shape" of a data file

Granularity -- how fine/coarse is each datum

Scope -- how (in)complete is the data

Temporality -- how is the data situated in time

Faithfulness -- how well does the data capture "reality"

CC BY NC SA

Variables are Columns

Let's look at records with the same granularity.

What does each **column** represent?

A **variable** is a **measurement** of a particular concept.

	U.S. jurisdiction	TB cases 2019	...
1	Alabama	87	...
2	Alaska	58	...
...	...	...	...

The U.S. Jurisdiction **variable**

It has two common properties:

- **Datatype/Storage type**:
  How each variable value is stored in memory. `df[colname].dtype`
  - integer, floating point, boolean, object (string-like), etc.

  Affects which pandas functions you use.
- **Variable type/Feature type**:
  Conceptualized measurement of information (and therefore what values it can take on).
  - Use expert knowledge
  - Explore data itself
  - Consult data codebook (if it exists).

  Affects how you visualize and interpret the data.

⚠️In this class, "variable types" are conceptual!!

CC BY NC SA

Variable Feature Types

Variable

Many variables do not fit neatly in one of these categories!!

Intervals have meaning.

Quantitative (numerical)

Qualitative (categorical)

Continuous

Discrete

Ordinal

Nominal

Could be measured to arbitrary precision.

Examples:
- Price
- Temperature

Finite possible values

Examples:
- Number of siblings
- Number of classes

Categories w/ordered levels; no consistent meaning to difference

Examples:
- Preferences
- Level of education

Categories w/ no specific ordering.

Examples:
- Political Affiliation
- Cal ID number

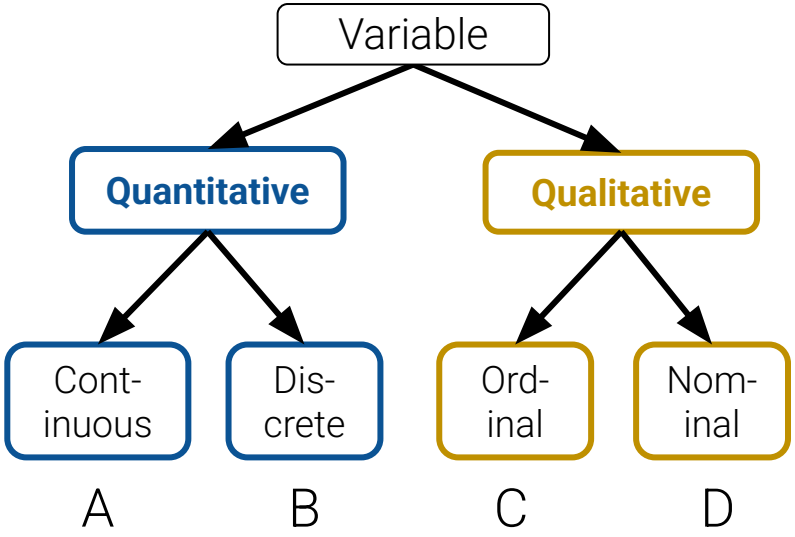
Note that **qualitative variables** could have numeric levels; conversely, **quantitative variables** could be stored as strings!

CC BY NC SA

Variable Types

What is the feature type (i.e., variable type) of each variable?

Q	Variable	Feature Type
1	CO_2 level (ppm)	
2	TB cases in 2021	
3	GPA	
4	Income bracket (low, med, high)	
5	Race/Ethnicity	
6	Number of years of education	
7	Yelp Rating	

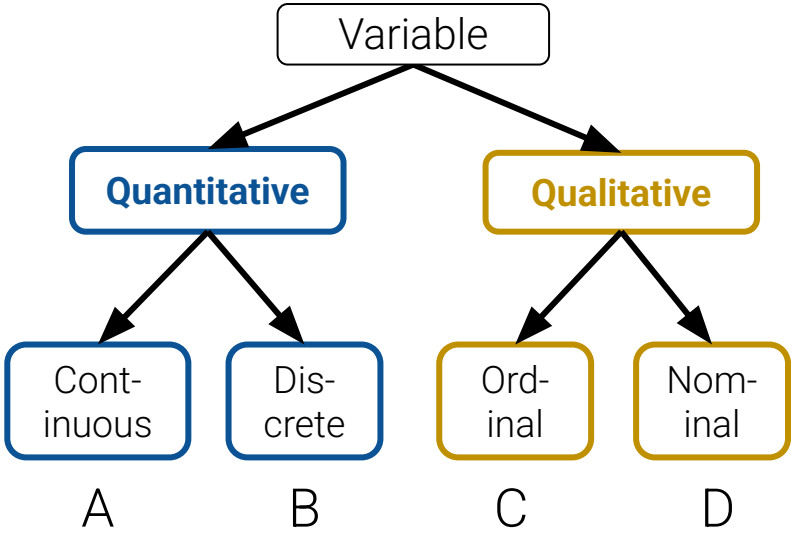


CC BY NC SA

Variable Types

What is the feature type of each variable?

Q	Variable	Feature Type
1	CO_2 level (ppm)	**A. Quantitative Cont.**
2	Number of siblings	**B. Quantitative Discrete**
3	GPA	**A. Quantitative Cont.**
4	Income bracket (low, med, high)	**C. Qualitative Ordinal**
5	Race/Ethnicity	**D. Qualitative Nominal**
6	Number of years of education	**B. Quantitative Discrete**
7	Yelp Rating	**C. Qualitative Ordinal**



Many of these examples show how "shaggy" these categories are!!
We will revisit variable types when we learn how to visualize variables.

CC BY NC SA